

MÁSTER EN INTELIGENCIA ARTIFICIAL APLICADA A LAS FINANZAS

Analizador Bursátil con IA

Análisis técnico y fundamental, recomendación por scoring determinista y asistente conversacional con grounding (RAG)

Memoria técnica del proyecto

Tipo de proyecto	Aplicación web académica de análisis bursátil
Repositorio	github.com/carmengata12697-dotcom/AUTOMATIZACION
Tecnologías núcleo	Python · Streamlit · yfinance · ta · Plotly · reportlab · Groq · pytest
Arquitectura	Separada por capas (Clean Architecture)
Equipo	5 integrantes (evaluación por historial de commits)
Fecha	19 de junio de 2026

Documento generado a partir del análisis directo del código fuente del repositorio.

Índice

1. Resumen ejecutivo.....	4
1.1 Características principales.....	4
2. Contexto y objetivos.....	5
2.1 Contextualización del problema.....	5
2.2 Objetivo general.....	5
2.3 Objetivos específicos.....	5
2.4 Requerimientos.....	5
3. Marco teórico y revisión de la literatura.....	7
3.1 Análisis técnico.....	7
3.2 Análisis fundamental.....	7
3.3 Modelos de scoring multicriterio.....	8
3.4 IA generativa con grounding: RAG.....	8
3.5 Arquitectura de software: Clean Architecture.....	8
4. Arquitectura del sistema.....	9
4.1 Diagrama de capas.....	9
4.2 Descripción de cada capa.....	9
4.3 Configuración central — config.py.....	10
5. El motor de scoring determinista.....	11
5.1 Criterios y pesos.....	11
5.2 Umbrales de recomendación.....	11
5.3 Tratamiento honesto de datos ausentes.....	11
6. Validación empírica del motor de scoring.....	13
6.1 Metodología.....	13
6.2 Resultados con los pesos base.....	13
6.3 Análisis de sensibilidad de los pesos.....	14
6.4 Conclusiones de la validación.....	14
7. El asistente conversacional (LLM + RAG).....	15
7.1 La regla de oro (grounding).....	15
7.2 El pipeline RAG.....	15
7.3 Interpretación de intención.....	15
7.4 Tolerancia a fallos.....	15
8. Instalación y uso.....	16
8.1 Puesta en marcha.....	16
8.2 Pruebas.....	16
8.3 Mercados soportados.....	16
9. Pruebas y aseguramiento de la calidad.....	17
10. Decisiones de diseño.....	18
11. Flujo de trabajo y trabajo en equipo.....	19

11.1 Convenciones de Git.....	19
11.2 Reparto de la contribución.....	19
12. Conclusiones y líneas futuras.....	20
12.1 Conclusiones.....	20
12.2 Líneas futuras.....	20
12.3 Descargo de responsabilidad.....	20
13. Referencias bibliográficas.....	21

1. Resumen ejecutivo

El Analizador Bursátil con IA es una aplicación web que, a partir de uno o varios símbolos bursátiles (tickers), descarga datos de mercado de Yahoo Finance, calcula indicadores técnicos y fundamentales, los puntúa mediante un motor de scoring determinista y emite una recomendación con semáforo (Comprar / Neutral / Evitar). Sobre estos resultados ya calculados opera un asistente conversacional basado en un modelo de lenguaje (LLM) que explica las cifras sin inventarlas.

El proyecto resuelve un problema concreto: el inversor individual y el analista deben procesar grandes volúmenes de información técnica y fundamental, dispersa en múltiples fuentes, antes de decidir sobre un activo. La aplicación centraliza, calcula e interpreta automáticamente esos indicadores y acompaña al usuario con una recomendación basada en reglas transparentes y ponderadas.

El valor académico y de ingeniería del proyecto reside en tres decisiones de diseño: (1) una arquitectura separada por capas que aísla la lógica de negocio de la interfaz y de las fuentes externas; (2) una recomendación 100 % determinista y auditable, no generada por el LLM; y (3) un asistente conversacional con grounding estricto —puede razonar sobre los números, pero tiene prohibido producirlos—. Esta separación garantiza trazabilidad, testabilidad y honestidad en los resultados.

1.1 Características principales

- **Análisis técnico:** RSI, MACD, Bandas de Bollinger y medias móviles (SMA 20/50/200, EMA 20) con gráficas interactivas de velas.
- **Análisis fundamental:** P/E, EPS, ROE, deuda/capital, margen neto y flujo de caja libre, normalizados a unidades coherentes.
- **Recomendación:** score de 0 a 100 con semáforo y desglose por indicador, mediante un motor de reglas con pesos transparentes.
- **Cartera:** comparación y ranking de hasta tres tickers a la vez.
- **Reporte PDF:** informe descargable con portada comparativa, gráficas, tablas y desglose del scoring.
- **Asistente (chatbot):** responde preguntas sobre el informe ya calculado, sin inventar cifras (sistema RAG con grounding).

Soporta tickers de EE. UU. y Latinoamérica (p. ej. AAPL, GOOGL, ECOPETROL.CL, WALMEX.MX, PETR4.SA).

2. Contexto y objetivos

2.1 Contextualización del problema

Los inversionistas individuales y los analistas financieros enfrentan el reto de procesar grandes volúmenes de información técnica y fundamental antes de tomar una decisión de compra o venta sobre un activo bursátil. La dispersión de esta información en múltiples fuentes, la complejidad de su interpretación conjunta y el tiempo requerido para su consolidación representan barreras reales para una toma de decisiones oportuna y fundamentada.

El proyecto propone una solución que centraliza, calcula e interpreta automáticamente los indicadores más relevantes de una acción cotizada, apoyando al usuario con una recomendación basada en reglas derivadas del análisis técnico y fundamental.

2.2 Objetivo general

Desarrollar una aplicación web interactiva que, a partir del símbolo bursátil de una acción, consulte datos desde Yahoo Finance, presente métricas técnicas y financieras organizadas por pestañas, y emita una recomendación argumentada mediante un modelo de scoring que pondera de forma transparente cada indicador.

2.3 Objetivos específicos

- Integrar la fuente de datos de Yahoo Finance (yfinance) para obtener históricos de precios, volumen e información financiera, con soporte para tickers de EE. UU. y Latinoamérica.
- Calcular y visualizar indicadores técnicos clave: RSI, MACD, Bandas de Bollinger y medias móviles, con ventana temporal configurable (1 mes, 3 meses, 1 año, 5 años).
- Extraer y presentar métricas fundamentales: P/E, EPS, ROE, deuda/capital, margen neto y flujo de caja libre.
- Implementar un motor de scoring que pondere ambas dimensiones y produzca una recomendación (Comprar / Neutral / Evitar).
- Desarrollar un módulo de comparación simultánea de hasta tres acciones, con ranking automático por score.
- Generar un reporte descargable en PDF con gráficas, tabla de scoring y justificación por indicador.
- Incorporar un asistente conversacional (LLM) que explique el informe ya calculado, con grounding estricto.
- Estructurar el código bajo principios de Clean Architecture y modularidad, con trazabilidad mediante GitHub.

2.4 Requerimientos

Requerimientos funcionales

- El usuario introduce hasta tres tickers con soporte para mercados de EE. UU. y Latinoamérica.
- La aplicación muestra cinco pestañas: Técnico, Fundamental, Recomendación, Cartera y Asistente.

- La pestaña técnica incluye gráficas interactivas de precio con indicadores superpuestos y ventana temporal configurable.
- La pestaña fundamental muestra tablas con los ratios financieros.
- La pestaña de recomendación presenta el scoring, la justificación por métrica y un semáforo visual.
- El módulo de cartera compara y ordena por score los tickers seleccionados.
- Generación y descarga de reporte PDF con gráficas, tabla de scoring y justificación.

Requerimientos no funcionales

- Interfaz operable desde navegador de escritorio (Streamlit, layout amplio).
- Código documentado, modular y versionado en GitHub con historial de commits por integrante.
- Cobertura de pruebas unitarias mínima del 70 % sobre los módulos de dominio (objetivo cumplido con holgura).
- La aplicación funciona íntegramente sin el chatbot; el LLM es una capa opcional.

3. Marco teórico y revisión de la literatura

Esta sección sustenta teóricamente las decisiones de diseño del proyecto. Cada indicador, ratio y patrón arquitectónico empleado se apoya en literatura reconocida del análisis técnico, del análisis fundamental y de la ingeniería de software y la IA generativa. Las referencias completas se recogen, en formato APA 7.^a edición, en la sección 13.

3.1 Análisis técnico

El análisis técnico estudia el comportamiento del precio y del volumen para anticipar movimientos futuros, partiendo de la premisa de que el precio descuenta toda la información disponible. El proyecto implementa cuatro familias de indicadores ampliamente validadas en la literatura.

Índice de Fuerza Relativa (RSI). Oscilador de momento propuesto por Wilder (1978), que mide la velocidad y magnitud de los cambios de precio en una escala de 0 a 100. Wilder fijó los umbrales convencionales de sobrecompra (70) y sobreventa (30) que el proyecto reutiliza, y definió el suavizado exponencial específico —hoy conocido como «media de Wilder»— que distingue al RSI de un promedio simple. El motor del proyecto considera saludable un RSI entre 30 y 70.

MACD (Moving Average Convergence Divergence). Indicador de tendencia y momento creado por Appel (2005), basado en la diferencia entre dos medias móviles exponenciales (12 y 26 periodos) y su línea de señal (EMA de 9 periodos). El cruce del MACD por encima de su señal se interpreta como impulso alcista, criterio que el proyecto adopta directamente.

Bandas de Bollinger. Envolventes de volatilidad descritas por Bollinger (2001), construidas a partir de una media móvil simple de 20 periodos más/menos dos desviaciones estándar. El proyecto emplea la banda inferior como señal de precio relativamente «barato» a corto plazo: cuando el precio toca o perfora dicha banda, el criterio suma su peso.

Medias móviles (SMA 200). La media móvil simple de 200 sesiones es un referente clásico de tendencia de largo plazo (Murphy, 1999): un precio por encima de la SMA 200 se asocia a una tendencia primaria alcista. El proyecto la usa como uno de sus criterios técnicos.

3.2 Análisis fundamental

El análisis fundamental valora una empresa a partir de sus estados financieros y su capacidad de generar valor. La doctrina moderna se remonta a Graham y Dodd (2009) y ha sido sistematizada para la valoración por Damodaran (2012). El proyecto selecciona cuatro métricas de uso extendido.

- **Relación precio/beneficio (P/E).** Múltiplo que relaciona el precio de la acción con el beneficio por acción. Damodaran (2012) lo describe como el múltiplo de valoración más utilizado; el proyecto considera atractivo un P/E entre 0 y 20, umbral fijo que sustituye —de forma transparente— a una mediana sectorial fiable no disponible en la fuente.
- **Rentabilidad sobre recursos propios (ROE).** Mide el beneficio generado por cada unidad de patrimonio. Un ROE positivo indica rentabilidad para el accionista (Graham & Dodd, 2009); el motor exige ROE mayor que cero.
- **Ratio deuda/capital.** Indicador de apalancamiento y salud financiera: a mayor ratio, mayor riesgo. El proyecto premia ratios inferiores a 1,5.

- **Flujo de caja libre (FCL).** Efectivo que la empresa genera tras cubrir sus inversiones; es la base de la valoración por descuento de flujos (Damodaran, 2012). Un FCL positivo es señal de solidez, y por su importancia el proyecto le asigna el mayor peso (18).

3.3 Modelos de scoring multicriterio

La combinación de múltiples señales en una única recomendación es un problema de decisión multicriterio. El motor del proyecto aplica un esquema de suma ponderada (weighted scoring model): cada criterio aporta su peso si se cumple, y el resultado se normaliza sobre los criterios efectivamente evaluables. Este enfoque, frente a un modelo de caja negra, prioriza la transparencia y la auditabilidad: cada punto del score es trazable hasta un criterio concreto y su umbral documentado.

3.4 IA generativa con grounding: RAG

El asistente conversacional se apoya en la técnica de Generación Aumentada por Recuperación (Retrieval-Augmented Generation, RAG), formalizada por Lewis et al. (2020). RAG combina un modelo de lenguaje con un componente de recuperación que aporta, en tiempo de inferencia, fragmentos de información relevante; el modelo redacta su respuesta condicionado a ese contexto en lugar de depender solo de su memoria paramétrica. Esta arquitectura reduce las «alucinaciones» y es precisamente la garantía que el proyecto necesita: el LLM razona sobre los números ya calculados, pero tiene prohibido inventarlos.

3.5 Arquitectura de software: Clean Architecture

La organización por capas con dependencias dirigidas hacia el dominio sigue los principios de la Arquitectura Limpia de Martin (2017): la lógica de negocio no depende de frameworks, interfaces ni fuentes externas, lo que la hace testeable y estable frente a cambios tecnológicos. En el proyecto, esto se traduce en puertas únicas a las fuentes externas (Yahoo Finance, Groq) y en un dominio puro que concentra el valor y las pruebas.

4. Arquitectura del sistema

La aplicación se estructura en capas independientes con una regla de dirección de dependencias clara: la interfaz depende del dominio, el dominio no depende de nada externo, y el acceso a fuentes externas está encapsulado en una única puerta de entrada por servicio. Esta separación es la base de la testabilidad y de la trazabilidad del proyecto.

4.1 Diagrama de capas

```

app.py (Streamlit: solo orquesta y presenta)
|
v
ui/  PRESENTACION
    tab_technical, tab_fundamental, tab_recommendation,
    tab_portfolio, tab_chat
|
v
domain/ LOGICA DE NEGOCIO PURA (testeable)
    technical_engine, fundamental_engine,
    scoring_engine (usa config.py),
    portfolio_engine (reservado)
|
v
data_layer/ UNICA puerta a Yahoo Finance
    yahoo_client --> yfinance --> Yahoo Finance

llm/  CAPA CONVERSACIONAL (opcional)
    groq_client (unica puerta al LLM),
    intent_parser, RAG (glosario + grounding),
    report_chat (punto de entrada, delega en RAG)

reports/pdf_generator PDF desde resultados ya calculados
  
```

4.2 Descripción de cada capa

Capa de datos — data_layer/

yahoo_client.py es la **única** puerta de entrada a Yahoo Finance. Ningún otro módulo importa `yfinance` ni llama a una API externa. Expone dos funciones puras: `obtener_historico()`, que devuelve un DataFrame de precios (Open, High, Low, Close, Volume) ya saneado —descarta días sin cierre, frecuentes en cotizaciones latinoamericanas—, y `obtener_fundamentales()`, que devuelve un diccionario limpio de métricas. Ante cualquier fallo devuelve estructuras vacías; nunca lanza excepción ni inventa datos.

Capa de dominio — domain/

Contiene la lógica de negocio pura, sin dependencias de framework ni de red. Es la capa testeable y el corazón del proyecto. Cuatro motores:

- **technical_engine.py** — calcula RSI, MACD, Bandas de Bollinger y medias móviles sobre la serie de cierres (con la librería `ta`). Devuelve series/tablas completas, útiles para graficar.

- **fundamental_engine.py** — normaliza las métricas crudas a unidades coherentes (por ejemplo, convierte la deuda/capital de escala porcentaje a ratio). Los valores ausentes se conservan como None.
- **scoring_engine.py** — núcleo determinista; combina señales técnicas y fundamentales con los pesos de config y produce el score 0-100, la recomendación y el desglose por indicador.
- **portfolio_engine.py** — reservado para una futura extracción de la lógica de cartera al dominio; hoy la comparación vive en la UI.

Capa de presentación — ui/

Cinco pestañas de Streamlit que solo presentan: llaman a los motores de dominio y a la capa de datos, y renderizan los resultados con Plotly. No contienen lógica de cálculo de negocio. La pestaña de cartera concentra hoy la comparación multi-ticker.

Capa de reportes — reports/

pdf_generator.py genera el informe PDF con reportlab y matplotlib a partir de resultados ya calculados: portada comparativa multi-ticker y una sección por acción con gráficas, tablas y el desglose del scoring.

Capa conversacional — llm/

Encapsula el chatbot. **groq_client.py** es la única puerta al proveedor LLM (Groq), igual que data_layer lo es a Yahoo. **intent_parser.py** traduce lenguaje natural a una cartera estructurada {ticker: peso}. **RAG.py** implementa el sistema de recuperación-aumentación-generación, y **report_chat.py** es el punto de entrada que delega en RAG.

4.3 Configuración central — config.py

Todos los parámetros que no son lógica de negocio viven centralizados en config.py: mercados de ejemplo, ventanas temporales, los pesos del scoring, los umbrales de cada criterio y de recomendación, el descargo de responsabilidad y el modelo del LLM. Esto permite ajustar el comportamiento sin tocar los motores.

5. El motor de scoring determinista

El motor de scoring es el componente que da valor de ingeniería al proyecto: produce una recomendación auditable, reproducible y explicable, sin intervención del modelo de lenguaje. El score es el porcentaje del peso de los criterios cumplidos sobre el peso de los criterios que efectivamente se han podido evaluar.

5.1 Criterios y pesos

Los ocho criterios suman 100 puntos, repartidos entre la dimensión técnica (42 %) y la fundamental (58 %). Los pesos viven en config.PESOS_SCORING, no incrustados en el motor.

Indicador	Categoría	Peso	Criterio para sumar el peso
RSI	Técnico	10	Entre 30 y 70
MACD	Técnico	12	MACD por encima de su señal
Precio > SMA 200	Técnico	12	Tendencia alcista de largo plazo
Banda baja Bollinger	Técnico	8	Precio $\leq$ banda baja (barato a c/p)
P/E vs sector	Fundamental	14	P/E entre 0 y 20
ROE positivo	Fundamental	14	ROE mayor que 0
Deuda/Capital	Fundamental	12	Ratio menor que 1,5
Flujo de caja libre	Fundamental	18	FCL positivo

5.2 Umbrales de recomendación

Score	Recomendación	Semáforo
≥ 65	Comprar	Verde
40 - 64	Neutral	Ámbar
< 40	Evitar	Rojo

5.3 Tratamiento honesto de datos ausentes

Una decisión clave: los criterios que no se pueden evaluar por falta de datos se excluyen del cálculo, y el score se renormaliza sobre los pesos realmente evaluados. No se penaliza una acción por datos que Yahoo no proporciona, ni se inventa un valor. El motor reporta además el "peso evaluado" (sobre 100), de modo que el usuario sabe con qué cobertura de información se ha calculado la recomendación. Si no hay ningún criterio evaluable, devuelve "Datos insuficientes" en lugar de un número engañoso.

Simplificaciones transparentes documentadas en el código: "P/E vs sector" usa un umbral fijo (Yahoo no ofrece una mediana sectorial fiable) y "ROE creciente" se reduce a comprobar que el ROE sea

positivo (sin histórico no se mide el crecimiento). Ambas son mejorables y están señaladas como tales.

6. Validación empírica del motor de scoring

Para comprobar que el motor de scoring produce resultados coherentes y robustos, se ejecutó sobre datos reales de mercado usando el código de dominio del propio proyecto (domain/scoring_engine.py) y los pesos de config.py, sin modificación alguna. Esta sección documenta tanto los resultados con los pesos base como un análisis de sensibilidad que pone a prueba la estabilidad del ranking ante cambios en la ponderación.

6.1 Metodología

- **Muestra:** tres valores de gran capitalización del mercado estadounidense (AAPL, MSFT y GOOGL), elegidos por disponer de datos técnicos y fundamentales completos.
- **Datos técnicos:** serie diaria de cierres del último año ($\approx$ 264 sesiones, junio 2025 – junio 2026). Sobre ella se calcularon, con las mismas fórmulas de la librería ta que usa el proyecto, el RSI (media de Wilder, 14), el MACD (12/26/9), la SMA 200 y la banda inferior de Bollinger ($20, 2\sigma$).
- **Datos fundamentales:** P/E, ROE, deuda/capital y flujo de caja libre del ejercicio FY2025.
- **Fuente:** datos de mercado de Yahoo Finance, obtenidos el 19 de junio de 2026 a través del proveedor financiero de Perplexity.
- **Procedimiento:** se invocó `calcular_score(técnico, fundamental, pesos)` por cada valor; el script reproducible es `backtest.py`, incluido en el repositorio.

6.2 Resultados con los pesos base

Con la ponderación oficial del proyecto, los tres valores fueron evaluados sobre el 100 % de los criterios (cobertura completa de datos). El resultado:

Ticker	Score	Recomendación	Criterios cumplidos (5/8)
AAPL	66,0	Comprar	RSI, precio>SMA200, ROE>0, deuda/capital, FCL
GOOGL	66,0	Comprar	RSI, precio>SMA200, ROE>0, deuda/capital, FCL
MSFT	54,0	Neutral	RSI, ROE>0, deuda/capital, FCL

Interpretación: AAPL y GOOGL alcanzan «Comprar» (66) al cumplir los cinco criterios de mayor peso, incluida la tendencia alcista de largo plazo (precio sobre la SMA 200). MSFT obtiene «Neutral» (54) porque, pese a unos fundamentales sólidos, su precio cotizaba por debajo de la SMA 200 y su MACD estaba por debajo de la señal en la fecha de corte —es decir, momento técnico desfavorable—. El resultado es financieramente razonable: el motor no premia a una empresa buena que atraviesa un tramo bajista de corto/medio plazo, exactamente el comportamiento esperado de un modelo que pondera técnico y fundamental.

6.3 Análisis de sensibilidad de los pesos

Un riesgo de todo modelo de scoring ponderado es que el resultado dependa en exceso de pesos elegidos de forma subjetiva. Para medirlo, se recalculó el score de los tres valores bajo cuatro esquemas de ponderación distintos, manteniendo intactos los datos y los umbrales:

- **Base (proyecto):** la ponderación oficial (técnico 42 / fundamental 58).
- **Igualitario:** 12,5 puntos por criterio (8 criterios).
- **Pro-técnico (60/40):** se prioriza la dimensión técnica.
- **Pro-fundamental (20/80):** se prioriza la dimensión fundamental.

Esquema de pesos	AAPL	MSFT	GOOGL
Base (proyecto)	66,0 · Comprar	54,0 · Neutral	66,0 · Comprar
Igualitario (12,5 c/u)	62,5 · Neutral	50,0 · Neutral	62,5 · Neutral
Pro-técnico (60/40)	60,0 · Neutral	45,0 · Neutral	60,0 · Neutral
Pro-fundamental (20/80)	70,0 · Comprar	65,0 · Comprar	70,0 · Comprar

6.4 Conclusiones de la validación

- **Ranking estable.** En los cuatro esquemas, AAPL y GOOGL empatan y se sitúan siempre por encima de MSFT. El orden relativo no se altera al cambiar la ponderación: la conclusión del modelo es robusta y no un artefacto de los pesos elegidos.
- **Sensibilidad acotada.** Las puntuaciones se mueven en un rango estrecho (AAPL/GOOGL entre 60 y 70; MSFT entre 45 y 65). Ningún valor cruza de «Comprar» a «Evitar» ni viceversa; los cambios de etiqueta se limitan a la frontera Comprar/Neutral, lo esperable cerca del umbral de 65.
- **Coherencia con la teoría.** El esquema pro-fundamental eleva a las tres empresas (todas tienen fundamentales sólidos), mientras que el pro-técnico las penaliza (MSFT y GOOGL arrastraban señales técnicas débiles en la fecha de corte). El comportamiento del motor responde a la lógica financiera descrita en el marco teórico.
- **Trazabilidad.** Todos los resultados se obtuvieron ejecutando el código de producción del proyecto sin modificarlo, lo que confirma que la implementación realiza exactamente lo que la documentación describe.

Limitaciones: la muestra es reducida (tres valores de un mismo mercado y sector tecnológico) y se evalúa un único corte temporal; la cobertura de fundamentales para mercados latinoamericanos en la fuente fue parcial. Estas limitaciones se recogen como líneas futuras en la sección 12.

7. El asistente conversacional (LLM + RAG)

El asistente permite preguntar en lenguaje natural sobre el informe ya calculado. Está construido como un sistema RAG (Retrieval-Augmented Generation) con una regla de oro que define toda la capa.

7.1 La regla de oro (grounding)

El LLM nunca calcula ni inventa. Puede razonar y relacionar los números que se le entregan, pero tiene prohibido producir cifras o juicios que no estén en el contexto. La recomendación Comprar / Neutral / Evitar sale siempre del scoring_engine, nunca del LLM, y siempre acompañada del descargo de responsabilidad.

7.2 El pipeline RAG

1. Recuperación: de todo lo calculado, RAG.py selecciona solo lo relevante a la pregunta — los tickers mencionados (por símbolo o nombre) y las definiciones del glosario cuyas palabras clave aparezcan en la pregunta.
2. Aumentación: construye el prompt añadiendo ese contexto recuperado, marcando explícitamente qué son datos del informe y qué son definiciones conceptuales.
3. Generación: el LLM (Groq) redacta la respuesta usando únicamente ese contexto, con temperatura baja (0,2) para favorecer fidelidad sobre creatividad.

El glosario solo aporta definiciones conceptuales (qué es el RSI, el P/E...), nunca datos de la acción, evitando que el modelo improvise valores de memoria.

7.3 Interpretación de intención

intent_parser.py traduce frases como "quiero 50 % en AAPL y el resto a partes iguales entre MSFT y GOOGL" a {AAPL: 0,5; MSFT: 0,25; GOOGL: 0,25}. Aquí "calcular" se limita a repartir los pesos que el propio usuario expresa; ningún dato de mercado sale del LLM. Si el LLM no está disponible, un extractor por expresiones regulares actúa como red de seguridad.

7.4 Tolerancia a fallos

Toda la capa está diseñada para no romper la aplicación: si falta la clave de Groq o la librería, el cliente devuelve un aviso claro y el resto de la app (análisis y recomendación) funciona con normalidad. El chatbot es estrictamente opcional.

8. Instalación y uso

8.1 Puesta en marcha

```
# 1. Entorno virtual
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate

# 2. Dependencias
pip install -r requirements.txt

# 3. (Opcional) Chatbot: clave de Groq
cp .env.example .env        # Windows: copy .env.example .env
# Clave gratuita en https://console.groq.com/keys

# 4. Arrancar
streamlit run app.py
```

Se abre el navegador con cinco pestañas: Técnico, Fundamental, Recomendación, Cartera y Asistente. El Asistente necesita la clave de Groq en .env; el resto de la aplicación funciona sin ella.

8.2 Pruebas

```
python -m pytest          # toda la suite
python -m pytest --cov=domain # con cobertura sobre los motores
```

8.3 Mercados soportados

Mercado	Ejemplo de ticker
Estados Unidos	AAPL, GOOGL, MSFT
Colombia	ECOPETROL.CL
México	WALMEX.MX
Brasil	PETR4.SA

9. Pruebas y aseguramiento de la calidad

Los motores de dominio se prueban con pytest y datos sintéticos, sin dependencias de red, lo que las hace rápidas y reproducibles. El objetivo no funcional era una cobertura mínima del 70 % sobre domain/; el proyecto la supera ampliamente (aprox. 95 %).

Módulo de pruebas	Qué verifica
test_technical.py	RSI en rango 0–100, SMA de serie constante, orden de las bandas de Bollinger, columnas del MACD y robustez ante DataFrame vacío.
test_fundamental.py	Conversión de deuda/capital a ratio, conservación de ROE y margen como fracción, valores ausentes como None y exclusión de bool como número.
test_scoring.py	Umbrales de clasificación, casos extremos (todo cumplido = 100, nada cumplido = 0), renormalización ante datos ausentes y estructura del desglose.
test_portfolio.py	Los pesos de la cartera suman uno.
test_llm.py	Normalización y reescalado de pesos, fallback por regex, formato de porcentajes y recuperación RAG por ticker y por concepto.

Detalle relevante: incluso la capa RAG se prueba de forma determinista —se verifica que la recuperación filtra por el ticker mencionado y que incorpora la definición correcta del glosario— sin necesidad de llamar al LLM real.

10. Decisiones de diseño

Estas decisiones están registradas en CLAUDE.md como acuerdos del grupo que no se revierten sin consenso. Son el aspecto que aporta mayor valor académico a la documentación.

Decisión	Justificación
Streamlit, no Flask	El repo arrancó con Flask; se descartó para centrarse en una interfaz de datos rápida de construir. Los archivos Flask se eliminan en un commit aparte y documentado, nunca en silencio.
Librería "ta", no "pandas-ta"	pandas-ta presentaba problemas de compatibilidad con versiones recientes de pandas/numpy; ta es estable.
Nombres en inglés sin acentos	Evita problemas de codificación de nombres de archivo entre Windows, Linux y macOS.
Pesos en config, no en el motor	Permite ajustar el scoring sin tocar la lógica; favorece la transparencia y la trazabilidad.
Groq como proveedor LLM	Modelos open-source gratuitos (p. ej. llama-3.3-70b-versatile) con cuota suficiente; toda la comunicación pasa por groq_client.py.
Recomendación 100 % determinista	La etiqueta Comprar/Neutral/Evitar sale del scoring, no del LLM: resultado auditable y reproducible.

11. Flujo de trabajo y trabajo en equipo

11.1 Convenciones de Git

- **Ramas:** main (estable) ← develop (integración) ← feature/* (una rama por funcionalidad), fusionadas mediante Pull Request.
- **Commits:** Conventional Commits (feat:, fix:, chore:, test:, docs:), descriptivos, frecuentes y repartidos por integrante.
- **Secretos:** la clave del LLM va en .env (ignorado por git); nunca se sube al repositorio.

11.2 Reparto de la contribución

La participación individual se evalúa por el historial de commits. El reparto observado en el repositorio refleja una construcción por fases, con cada integrante responsable de un bloque del sistema:

Integrante (autor Git)	Commits	Área principal aportada
Alonsovr	19	Capa de datos, motores técnico/fundamental/scoring, limpieza, integración (merges) y README final.
Javier Montaner	8 (+1)	Toda la capa LLM: cliente Groq, parser de intención, RAG y pestaña del asistente.
Andrés Felipe Castro Mejía	7	Soporte multi-ticker, pestaña de cartera y generador de reporte PDF.
Tatiana750	6	Pestañas técnica, fundamental y de recomendación; sidebar y botón de descarga PDF.
carmengata12697	4	Base inicial Streamlit e integración del primer Pull Request.

Nota: los nombres corresponden a los autores configurados en Git. Conviene verificar que cada integrante committee con su identidad real, ya que la evaluación individual depende de ello.

12. Conclusiones y líneas futuras

12.1 Conclusiones

- El proyecto cumple sus objetivos: integra datos reales de mercado, calcula indicadores técnicos y fundamentales, emite una recomendación argumentada y la explica en lenguaje natural.
- La arquitectura por capas, con puertas únicas a Yahoo y a Groq, demuestra una aplicación rigurosa de los principios de Clean Architecture.
- La recomendación determinista y el grounding del LLM resuelven el principal riesgo de los asistentes financieros con IA: la invención de cifras.
- La cobertura de pruebas (~95 % sobre domain/) supera holgadamente el objetivo del 70 % y respalda la fiabilidad de los motores.

12.2 Líneas futuras

- Extraer la lógica de cartera de la UI al portfolio_engine, añadiendo composición ponderada y métricas de conjunto (correlación, diversificación, riesgo agregado).
- Incorporar la mediana sectorial real para el criterio P/E y el histórico de ROE para medir crecimiento.
- Persistir el histórico de búsquedas en sesión y ampliar la cobertura de mercados latinoamericanos.
- Añadir pruebas de integración de la UI y validación automática de los datos de la capa data_layer.

12.3 Descargo de responsabilidad

Esta recomendación es orientativa y se genera de forma automática a partir de reglas predefinidas. No constituye asesoría financiera profesional.

13. Referencias bibliográficas

Referencias citadas en el marco teórico y en la validación empírica, en formato APA 7.ª edición.

Análisis técnico y fundamental

Appel, G. (2005). *Technical analysis: Power tools for active investors*. FT Prentice Hall.

Bollinger, J. (2001). *Bollinger on Bollinger Bands*. McGraw-Hill.

Damodaran, A. (2012). *Investment valuation: Tools and techniques for determining the value of any asset* (3.ª ed.). John Wiley & Sons.

Graham, B., & Dodd, D. L. (2009). *Security analysis* (6.ª ed.). McGraw-Hill. (Obra original publicada en 1934)

Murphy, J. J. (1999). *Technical analysis of the financial markets: A comprehensive guide to trading methods and applications*. New York Institute of Finance.

Wilder, J. W. (1978). *New concepts in technical trading systems*. Trend Research.

Inteligencia artificial e ingeniería de software

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. En *Advances in Neural Information Processing Systems* (Vol. 33, pp. 9459–9474). Curran Associates.
<https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Herramientas y bibliotecas

Padroni, D. (2018). *ta: Technical Analysis Library in Python* [Software]. <https://github.com/bukosabino/ta>

Streamlit Inc. (2024). *Streamlit documentation*. <https://docs.streamlit.io>

yfinance. (2024). *yfinance: Download market data from Yahoo! Finance's API* [Software].
<https://github.com/ranaroussi/yfinance>

Los datos de mercado empleados en la validación empírica (sección 6) se obtuvieron de Yahoo Finance el 19 de junio de 2026.